DA3408 A3: Spark vs. Ray — The Data Engineering Duel

Benchmark Report

This report compares two implementations of the same NYC Yellow Taxi preprocessing workflow using PySpark and Ray Data. The benchmark uses the real multi-month TLC dataset rather than synthetic data and measures end-to-end execution time together with process-level resource observations.

1. Dataset and Pipeline

Dataset: 24 monthly NYC Yellow Taxi Parquet files covering January 2023 through December 2024, with 79,479,946 input rows. A 265-row taxi-zone lookup CSV provides location metadata.

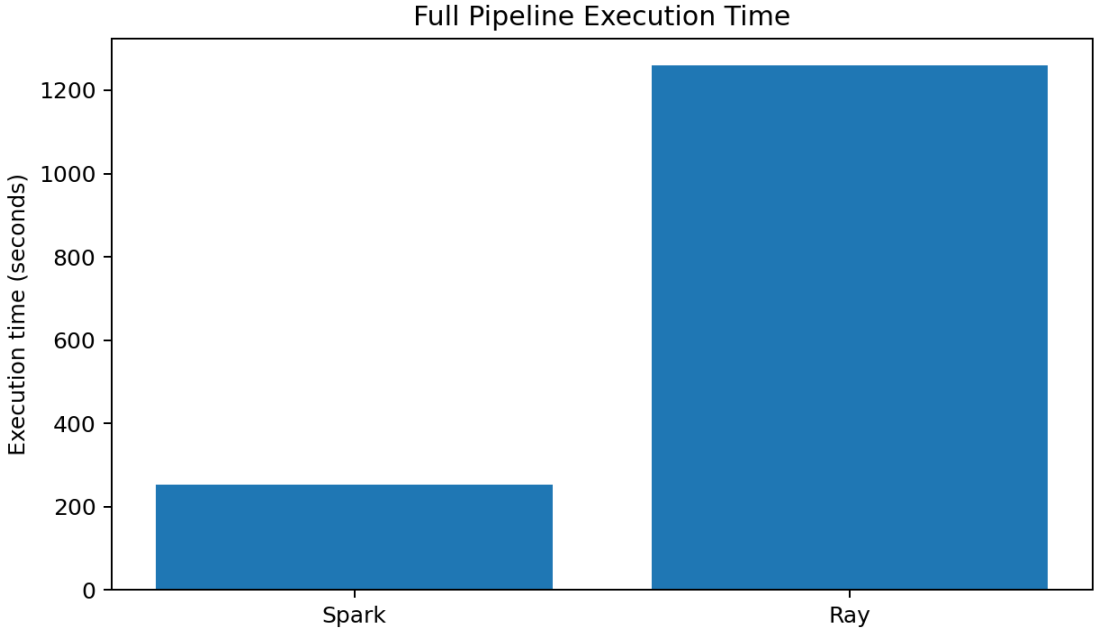
Pipeline: ingestion → required-field cleansing → duplicate removal → timestamp-based trip-duration calculation → average-speed calculation using a Python UDF/function → pickup/drop-off location enrichment → structured column selection → Parquet export.

2. Cluster Configuration

Framework	Configuration	Verification
Spark	1 master + 2 worker containers	Spark Master UI: 2 alive workers
Ray	1 head + 2 worker nodes	Ray Dashboard: 3 alive nodes

3. Benchmark Results

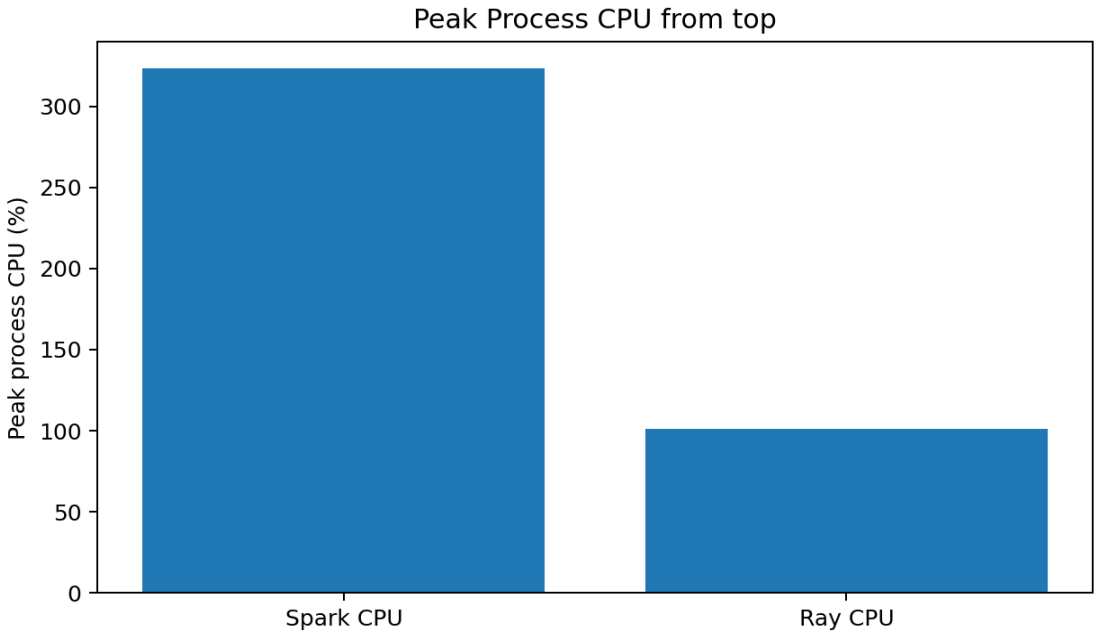
Metric	Spark	Ray
Full execution time	252.970 s	1260.419 s
Equivalent time	4m 12.97s	21m 00.42s
Relative runtime	1.00×	4.98×
Peak process CPU	323.5%	101.3%
Peak process memory	16.4%	11.6%
Peak process RES	~1.9 GB	~1.4 GB

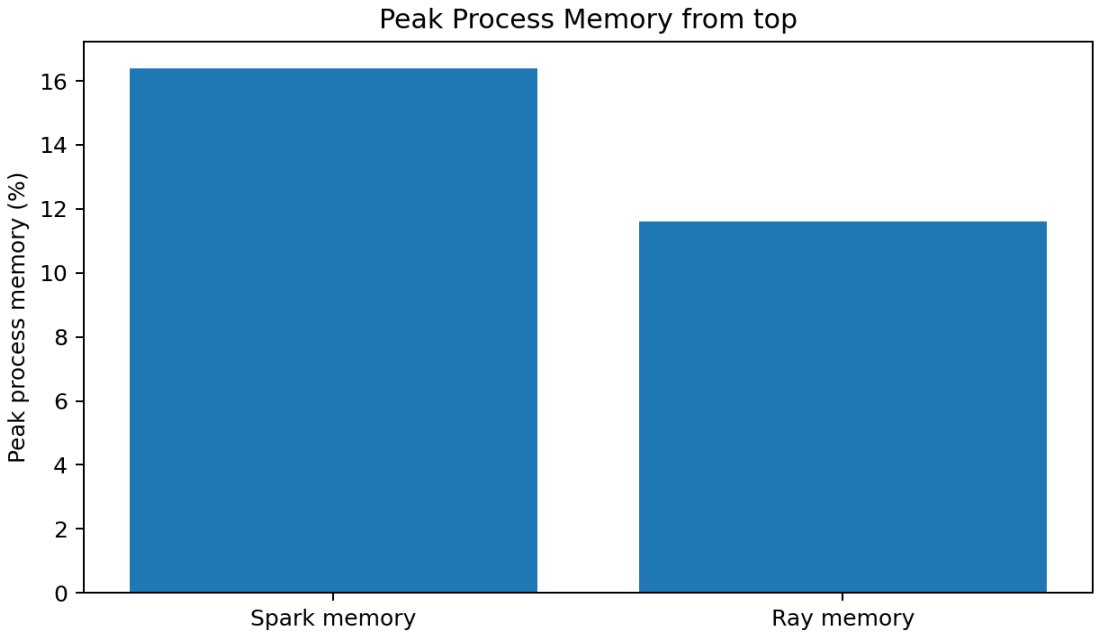


Spark completed the complete benchmark in 252.970 seconds, while Ray required 1260.419 seconds. Thus Ray took approximately 4.98 times as long under the measured configuration.

4. Resource Observations

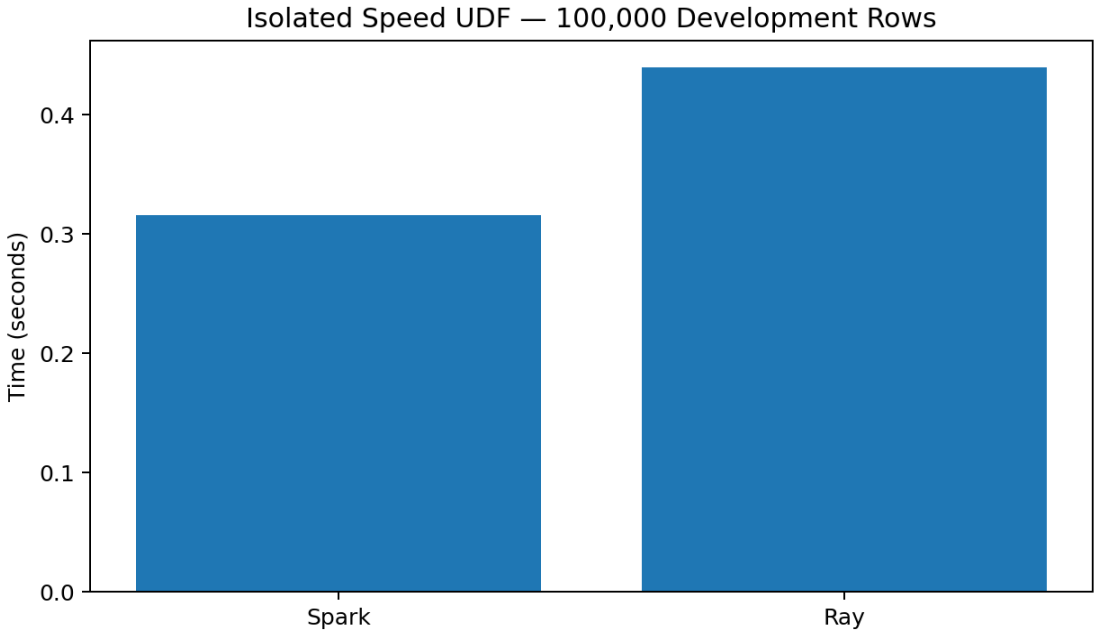
CPU and memory figures below are process-level peaks observed with Linux *top*. They should not be interpreted as total cluster-wide utilization. Spark's peak CPU sample was 323.5% and its peak memory sample was 16.4%; Ray's corresponding process-level peaks were 101.3% and 11.6%. The Spark and Ray runs therefore show different resource profiles, but the dominant result for this workload is the large end-to-end runtime difference.





5. UDF Deep-Dive

The average-speed calculation is the Python-defined transformation examined for the UDF discussion. A separate isolated experiment was run on 100,000 rows from the real development subset so that the speed calculation itself could be timed without another full 79.5M-row benchmark.



Measured development-scale UDF timing: Spark 0.316 s; Ray 0.440 s for 100,000 rows. The result is intentionally reported as a development-scale isolated experiment, not as a full-dataset UDF timing. Ray's Python-native model avoids Spark's JVM-to-Python UDF boundary in architecture, but Python-native execution does not guarantee lower wall-clock time. In this controlled measurement Spark was slightly faster.

6. Parity and Performance-Tuning Notes

The high-level preprocessing intent is shared by both implementations, but the internal execution strategies are not perfectly identical. Spark uses relational joins against the location lookup, while the final Ray implementation uses the small 265-row lookup as an in-memory dictionary. A distributed Ray join was tested but produced severe memory pressure on the constrained VM. This optimization was retained to make the full real-data run practical and is explicitly disclosed.

Duplicate handling also differs internally: Spark performs duplicate removal within each monthly input file, while Ray performs batch-level duplicate removal. This limitation is documented rather than hidden. The benchmark therefore compares the practical implementations executed on the same real dataset rather than claiming mathematically identical internal execution plans.

7. Framework Choice and Winner

BI-first / data-engineering workload: Spark is the clear winner in this experiment. Its 252.970-second end-to-end runtime was approximately 5× faster than Ray's 1260.419 seconds. The result is specific to the tested machine, cluster configuration and pipeline and should not be interpreted as a universal ranking of the frameworks.

AI-first workload: Ray remains attractive for Python-native distributed computation and workloads that combine data processing with Python-heavy or AI-oriented tasks. Its execution model can reduce friction when the surrounding workload is already Python-native. That architectural advantage did not translate into a runtime win for this particular taxi-data benchmark.

WINNER: SPARK

For the measured DA3408 A3 workload, Spark provides the best observed end-to-end performance.

8. Reproducibility and Evidence

The repository includes the Spark and Ray pipeline scripts, benchmark results, documentation, AI disclosure, and captured cluster/application evidence. The Spark evidence includes the Master UI and application Jobs/Stages views. The Ray evidence includes the Dashboard Cluster view and runtime cluster state. The reported benchmark values come from actual executions; no synthetic performance values were used.